

Regular Expressions

Regular Languages

- A language L is called a **regular language** if there is a DFA or an NFA for L .
- **Theorem:** The following are equivalent:
 - L is a regular language.
 - There is a DFA D where $\mathcal{L}(D) = L$.
 - There is an NFA N where $\mathcal{L}(N) = L$.
- In other words, knowing any one of the above three facts means you know the other two.

Language Concatenation

- If $w \in \Sigma^*$ and $x \in \Sigma^*$, then wx is the **concatenation** of w and x .
- If L_1 and L_2 are languages over Σ , the **concatenation** of L_1 and L_2 is the language L_1L_2 defined as

$$L_1L_2 = \{ x \mid \exists w_1 \in L_1. \exists w_2 \in L_2. x = w_1w_2 \}$$

- Example: if $L_1 = \{ a, ba, bb \}$ and $L_2 = \{ aa, bb \}$, then

$$L_1L_2 = \{ aaa, abb, baaa, babb, bbba, bbbb \}$$

Lots and Lots of Concatenation

- Consider the language $L = \{ \text{aa}, \text{b} \}$
- LL is the set of strings formed by concatenating pairs of strings in L .

$\{ \text{aaaa}, \text{aab}, \text{baa}, \text{bb} \}$

- LLL is the set of strings formed by concatenating triples of strings in L .

$\{ \text{aaaaaaa}, \text{aaaab}, \text{aabaa}, \text{aabb}, \text{baaaa}, \text{baab}, \text{bbaa}, \text{bbb} \}$

- $LLLL$ is the set of strings formed by concatenating quadruples of strings in L .

$\{ \text{aaaaaaaa}, \text{aaaaaab}, \text{aaaabaa}, \text{aaaabb}, \text{aabaaaa}, \text{aabbaab}, \text{aabbaa}, \text{aabbb}, \text{baaaaaa}, \text{baaaab}, \text{baabaa}, \text{baabb}, \text{bbaaaa}, \text{bbaab}, \text{bbbaa}, \text{bbbb} \}$

Language Exponentiation

- We can define what it means to “exponentiate” a language as follows:

$$L^0 = \{\varepsilon\} \qquad L^{n+1} = LL^n$$

- So, for example, $\{ \text{aa}, \text{b} \}^3$ is the language

$$\{ \text{aaaaaa}, \text{aaaab}, \text{aabaa}, \text{aabb}, \\ \text{baaaa}, \text{baab}, \text{bbaa}, \text{bbb} \}$$

The Kleene Closure

- An important operation on languages is the ***Kleene Closure***, which is defined as

$$L^* = \{ w \in \Sigma^* \mid \exists n \in \mathbb{N}. w \in L^n \}$$

- Mathematically:

$$w \in L^* \quad \text{iff} \quad \exists n \in \mathbb{N}. w \in L^n$$

- Intuitively, all possible ways of concatenating zero or more strings in L together, possibly with repetition.

The Kleene Closure

If $L = \{ \text{a}, \text{bb} \}$, then $L^* = \{$

$\epsilon,$

$\text{a}, \text{bb},$

$\text{aa}, \text{abb}, \text{bba}, \text{bbbb},$

$\text{aaa}, \text{aabb}, \text{abba}, \text{abbbb}, \text{bbaa}, \text{bbabb}, \text{bbbba}, \text{bbbbbb},$

$\dots$

$\}$

Think of L^* as the set of strings you can make if you have a collection of stamps – one for each string in L – and you form every possible string that can be made from those stamps.

Closure Properties

- ***Theorem:*** If L_1 and L_2 are regular languages over an alphabet Σ , then so are the following languages:
 - $L_1 \cup L_2$
 - $L_1 L_2$
 - L_1^*
- These (and other) properties are called ***closure properties of the regular languages***.

Rethinking Regular Languages

- We currently have several tools for showing a language L is regular:
 - Construct a DFA for L .
 - Construct an NFA for L .
 - Combine several simpler regular languages together via closure properties to form L .
- We have not spoken much of this last idea.

Constructing Regular Languages

- **Idea:** Build up all regular languages as follows:
 - Start with a small set of simple languages we already know to be regular.
 - Using closure properties, combine these simple languages together to form more elaborate languages.
- *This is a bottom-up approach to the regular languages.*

Regular Expressions

- ***Regular expressions*** are a way of describing a language via a string representation.
- They're used just about everywhere:
 - They're built into the JavaScript language and used for data validation.
 - They're used in the UNIX grep and flex tools to search files and build compilers.
 - They're employed to clean and scrape data for large-scale analysis projects.
- Conceptually, regular expressions are strings describing how to assemble a larger language out of smaller pieces.

Atomic Regular Expressions

- The regular expressions begin with three simple building blocks.
- The symbol $\emptyset$ is a regular expression that represents the empty language $\emptyset$.
- For any $a \in \Sigma$, the symbol a is a regular expression for the language $\{a\}$.
- The symbol ϵ is a regular expression that represents the language $\{\epsilon\}$.
 - **Remember:** $\{\epsilon\} \neq \emptyset!$
 - **Remember:** $\{\epsilon\} \neq \epsilon!$

Compound Regular Expressions

- If R_1 and R_2 are regular expressions, R_1R_2 is a regular expression for the *concatenation* of the languages of R_1 and R_2 .
- If R_1 and R_2 are regular expressions, $R_1 \cup R_2$ is a regular expression for the *union* of the languages of R_1 and R_2 .
- If R is a regular expression, R^* is a regular expression for the *Kleene closure* of the language of R .
- If R is a regular expression, (R) is a regular expression with the same meaning as R .

Operator Precedence

- Here's the operator precedence for regular expressions:

(R)

R^*

R_1R_2

$R_1 \cup R_2$

- So **ab*cUd** is parsed as **((a(b*))c)Ud**

Regular Expression Examples

- The regular expression **trickUtrear** represents the language

{ **trick**, **trear** }.

- The regular expression **booo*** represents the regular language

{ **boo**, **booo**, **boooo**, ... }.

- The regular expression **candy!(candy!)*** represents the regular language

{ **candy!**, **candy!candy!**, **candy!candy!candy!**, ... }.

Regular Expressions, Formally

- The *language of a regular expression* is the language described by that regular expression.
- Formally:
 - $\mathcal{L}(\epsilon) = \{\epsilon\}$
 - $\mathcal{L}(\emptyset) = \emptyset$
 - $\mathcal{L}(a) = \{a\}$
 - $\mathcal{L}(R_1 R_2) = \mathcal{L}(R_1) \mathcal{L}(R_2)$
 - $\mathcal{L}(R_1 \cup R_2) = \mathcal{L}(R_1) \cup \mathcal{L}(R_2)$
 - $\mathcal{L}(R^*) = \mathcal{L}(R)^*$
 - $\mathcal{L}((R)) = \mathcal{L}(R)$

Worthwhile activity: Apply this recursive definition to

$a(b \cup c)((d))$

and see what you get.

Designing Regular Expressions

- Let $\Sigma = \{a, b\}$.
- Let $L = \{ w \in \Sigma^* \mid w \text{ contains } aa \text{ as a substring} \}$.

$(a \cup b)^*aa(a \cup b)^*$

bbabbb**aa**bab

aaaa

bbbbbbabbbb**aa**bbbbbb

Designing Regular Expressions

- Let $\Sigma = \{a, b\}$.
- Let $L = \{ w \in \Sigma^* \mid w \text{ contains } aa \text{ as a substring} \}$.

$\Sigma^*aa\Sigma^*$

bbabbb**aa**bab

aaaa

bbbbbbabbbb**aa**bbbbbb

Designing Regular Expressions

- Let $\Sigma = \{\mathbf{a}, \mathbf{b}\}$.
- Let $L = \{ w \in \Sigma^* \mid |w| = 4 \}$.

Designing Regular Expressions

- Let $\Sigma = \{\mathbf{a}, \mathbf{b}\}$.
- Let $L = \{ w \in \Sigma^* \mid |w| = 4 \}$.

Σ^4

aaaa
baba
bbbb
baaa

Designing Regular Expressions

- Let $\Sigma = \{a, b\}$.
- Let $L = \{ w \in \Sigma^* \mid w \text{ contains at most one } a \}$.

$b^*a?b^*$

bbbbabbb

bbbbbb

abbb

a

Shorthand Summary

- R^n is shorthand for $RR \dots R$ (n times).
 - Edge case: define $R^0 = \varepsilon$.
- Σ is shorthand for “any character in Σ .”
- $R?$ is shorthand for $(R \cup \varepsilon)$, meaning “zero or one copies of R .”
- R^+ is shorthand for RR^* , meaning “one or more copies of R .”